



ASTR-SRv6 on PolarFly

Collective Routing with Multiple Spanning trees

Chris Martin

Principal Engineer, AI/Hyperscale Product

martincj@cisco.com

May 15, 2026

Companion Outline

1. Prologue: PolarFly in Seven Slides
2. Collective Communication and Parallelism
3. All Reduce on Disjoint Trees
4. All-Reduce on Multiple Spanning Trees
5. ASTR-SRv6: Collective Routing on PolarFly

Chapter 1

Prologue: PolarFly in Seven Slides



Why PolarFly, in One Slide

AI training is **all-reduce-heavy**: short paths and rich bisection per fabric hop matter. A **low-diameter** topology keeps collective latency and congestion down as GPU count grows.

PolarFly in one line

Prime q ; $N = q^2 + q + 1$ switches; fabric **diameter 2**; **near-Moore** (within a few % of the Hoffman–Singleton bound for $d=2$).

Takeaway

One parameter family, fixed small diameter, structured adjacency—ideal for tight collective schedules and for SRv6 encoding in the data plane.

Full proofs, routing, and cost models live in the main **PolarFly** tutorial; this talk assumes the facts below.

Modular Arithmetic & $\mathbb{F}_q$ Reminder

For prime q , the field $\mathbb{F}_q$ is $\{0, 1, \dots, q-1\}$ with $+$ and $\times \text{ mod } q$. Every $a \neq 0$ has a multiplicative inverse (Fermat: $a^{q-1} \equiv 1$, so $a^{-1} \equiv a^{q-2} \pmod{q}$).

Left-normalization convention (throughout)

Vectors in $\mathbb{F}_q^3$ are only defined up to a nonzero *left* scale factor $\lambda \in \mathbb{F}_q^\times$; we **left-normalize** to fix representatives when needed.

Worked example: $q = 7$

Find $3^{-1} \pmod{7}$:

$$3 \times 5 = 15 = 2 \times 7 + 1 \equiv 1.$$

So $3^{-1} \equiv 5$ in $\mathbb{F}_7$.

(Check: $3 \times 5 = 15 \equiv 1$.)

Projective Plane $\text{PG}(2, q)$ Reminder

Points. Nonzero $\mathbf{u} \in \mathbb{F}_q^3$ up to scale: $[\mathbf{u}] = [\lambda \mathbf{u}]$ for $\lambda \in \mathbb{F}_q^\times$ (left scaling).

Count

$$\# \text{points} = \frac{q^3 - 1}{q - 1} = q^2 + q + 1$$

One often left-normalizes (e.g. first nonzero coordinate = 1) to get a unique representative.

Small example: $q = 3$

$N = 3^2 + 3 + 1 = 13$ projective points.
(Fano-style bookkeeping; the four chapters all use the same point-of-view.)

The Graph ER_q Reminder

Vertices: points of $PG(2, q)$. Edge: $[u] \sim [v]$ iff $\langle u, v \rangle \equiv 0 \pmod{q}$.

- Degree q or $q + 1$ (one “quadric” type; full detail in the main presentation)
- Diameter 2 for all prime q
- C_4 -free (no 4-cycles) $\Rightarrow$ strong girth; girth 5

Erdős–Rényi / polarity

A classical *polarity graph*:
orthogonality mod q is the link rule.
This is the mathematical object
PolarFly instantiates in silicon.

SRv6 + uSID on PolarFly (Recap)

- RFC 8986 (SRv6): segment routing in the IPv6 destination, *network programming* in the data plane.
- Compressed uSID (e.g. RFC 9800 class of drafts): short 128-bit-friendly behavior codes; fits hardware fast paths on spine/leaf.
- Existing PolarFly profiles (tutorial constants):
`End.PF.FWD = 0xF020; End.PF.FLOOD = 0xF021; End.PF.TREE2 = 0xF022.`

ASTR adds three (this deck)

`End.ASTR.RootReplicate (0xF030); End.ASTR.Path (0xF031); End.ASTR.Egress (0xF032).`

The Collective-Routing Problem

What we are solving

Schedule an all-reduce so traffic uses **several edge-disjoint spanning trees** in ER_q , routing each tree's flow with SRv6, to **scale aggregate bandwidth** with the number of trees.

Why it fits PolarFly: diameter 2 and structured spreads/lines supply a mathematically *designed* family of trees (Singer/projective geometry), not a heuristic overlay.

Headline (preview of Chapters 13–14)

With $k = \left\lfloor \frac{q+1}{2} \right\rfloor$ trees, one can aim for **kx** the single-tree all-reduce bandwidth in the ideal pipelined regime (details and assumptions in the chapters).

Roadmap for This Talk

A linear progression (each chapter in the main deck, excerpted here):

1. **Collectives & parallelism** — models, cost, and why single-tree is not enough on a high-degree fabric (Ch. 12).
2. **Multi-tree math** — spreads, lines in $PG(2, q)$, and how Singer cycles yield disjoint objects to hang trees on (Ch. 13).
3. **Multi-tree all-reduce** — scheduling the reduce/ broadcast pattern across the trees; bandwidth scaling (Ch. 14).
4. **ASTR-SRv6** — `0xF030-0xF032` profiles, packet path, and how control plane data ties to the math (Ch. 15).

Read the prologue as “everything you need”; the four chapters are unchanged from the main PolarFly presentation build.

Chapter 2

Collective Communication and Parallelism



Why Collectives, and Why Now

- Chapters 3–11 treated PolarFly as a *graph*: routing, diameter, SRv6 plumbing.
- Real workloads — distributed training, HPC simulation, graph analytics — are **collective**: the entire cluster participates in one communication pattern.
- Wall-clock time is set by the *slowest* participant, not the average. Mis-designing a collective can burn more time than mis-designing the fabric.
- Next three chapters chase **bandwidth-optimal** All-Reduce on PolarFly using multiple edge-disjoint trees.

Roadmap

1. This chapter: vocabulary + α - β - γ cost model + Patarasuk–Yuan lower bound.
2. Next chapter: Singer representation & tree packing.
3. Then: the SPAA'23 multi-tree All-Reduce.
4. Then: ASTR-SRv6 (three profiles) that expresses it on the wire.

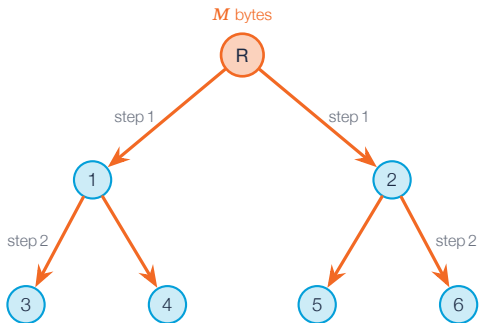
The Six Canonical Collectives

Collective	What it does
Broadcast	Root sends one buffer to everyone.
Reduce	Everyone contributes; root receives the reduction.
All-Reduce	Everyone contributes; <i>everyone</i> receives the reduction. Dominant op in data-parallel training.
All-Gather	Everyone contributes a slice; everyone receives the concatenation. Used in FSDP.
Reduce-Scatter	Everyone contributes; each node receives <i>one slice</i> of the reduction. Dual of All-Gather.
All-to-All	Every pair exchanges a distinct message. FFTs, sorting, MoE expert routing.

The ring decomposition

AllReduce $\equiv$ ReduceScatter + AllGather. Each half moves $\sim M(N-1)/N$ bytes per node; this is why ring All-Reduce is called “bandwidth-optimal” on a uniformly connected graph.

Broadcast



One root, one buffer, all destinations.

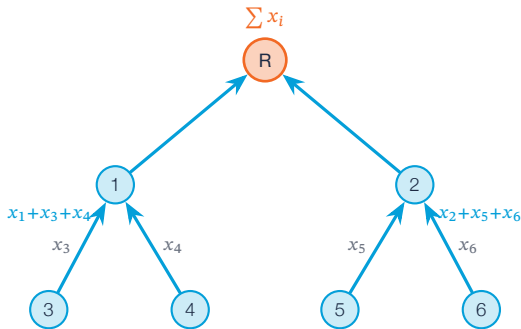
Root sends M bytes to every other node. Two strategies:

- **Tree broadcast:** root fans out in $\lceil \log_2 N \rceil$ steps — each node forwards as soon as it receives. Best for **small** messages (latency-bound).
- **Pipelined broadcast:** buffer split into $N-1$ chunks, streamed along a chain. Best for **large** messages (bandwidth-bound).

Where you see this

Parameter-server training: server broadcasts updated weights to all workers after each step. Also: checkpoint distribution, KV-cache pre-fill in inference serving.

Reduce



Everyone contributes; root gets the answer.

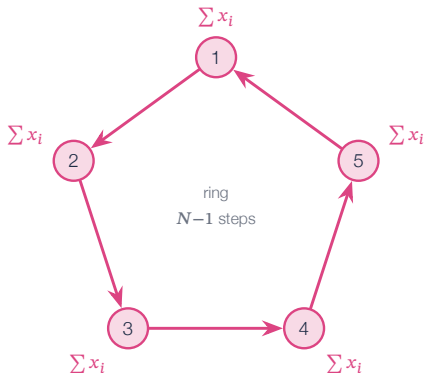
An element-wise operation (sum, max, min) across N input buffers. Only the **root** holds the final result.

- **Tree reduce:** $\lceil \log_2 N \rceil$ steps — partial reductions at internal nodes. Best for latency.
- **Pipeline reduce:** $N-1$ steps of M/N bytes flowing toward the root. Best for throughput.
- **Dual of Broadcast** — same tree, reversed arrows.

Where you see this

Centralized parameter-server gradient aggregation: workers send local gradients to the server, which sums them. Also: global loss / metric computation.

All-Reduce



Everyone contributes; **everyone** gets the result.

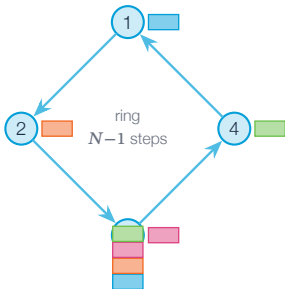
Logically = Reduce + Broadcast, fused via the *ring* decomposition:

- **Phase 1 — Reduce-Scatter:** $N-1$ steps, each node accumulates partial sums into its slice.
- **Phase 2 — All-Gather:** $N-1$ steps, each node broadcasts its reduced slice around the ring.
- **Bandwidth-optimal** on any Hamiltonian graph. Total data moved: $2M(N-1)/N$.

Where you see this

The dominant collective in data-parallel training. PyTorch DDP, DeepSpeed ZeRO-1 [1]: every GPU computes local gradients, All-Reduce sums them, every replica updates with the identical averaged gradient.

All-Gather



result: full vector at every node

Each node contributes a slice; everyone gets the whole.

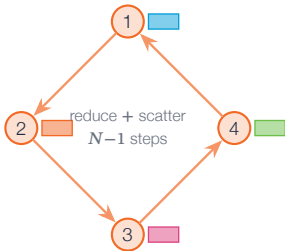
Each rank i holds shard s_i . After All-Gather every rank holds the full concatenation $[s_0, s_1, \dots, s_{N-1}]$.

- **Ring:** $N-1$ steps, each forwarding one shard to the next node.
- **No reduction** — pure data movement (copy, not sum).
- Dual of Reduce-Scatter. Second phase of ring All-Reduce.

Where you see this

FSDP / ZeRO-3: parameters are sharded across ranks. Before each forward pass, AllGather reconstructs the full parameter tensor. Also: gathering sharded activations for tensor-parallel layers.

Reduce-Scatter



After: node i holds reduced slice i only

Everyone contributes; each gets one reduced slice.

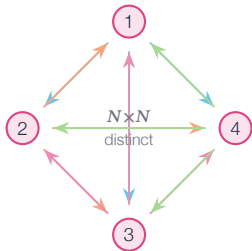
Every rank starts with a full buffer of M bytes. After Reduce-Scatter, rank i holds only the i -th slice of the reduced result — M/N bytes.

- Ring: $N-1$ steps, accumulating partial sums into each destination rank's slice.
- Dual of All-Gather. First phase of ring All-Reduce.
- Memory-efficient: each rank stores only $1/N$ of the result.

Where you see this

FSDP / ZeRO-2 gradient reduction: each rank reduces *only its own parameter shard's gradient*, avoiding the memory cost of a full All-Reduce. Also: the reduce phase of ring All-Reduce internally.

All-to-All



Every pair exchanges a unique payload

Personalized exchange: unique message per pair.

Each rank i sends a different buffer B_{ij} to every rank j .

No reduction — pure permutation / transpose of data.

- **Most communication-intensive:** $N(N-1)$ distinct messages, every link carries traffic.
- No ring shortcut — scheduling is the bottleneck.
- PolarFly advantage: diameter 2 means every pair is at most 2 hops apart, completing in $O(N/k)$ steps.

Where you see this

Mixture-of-Experts (MoE) routing: tokens dispatched to the rank hosting the assigned expert, then results sent back.

Also: distributed FFTs, parallel sorting, tensor resharding across mesh axes.

The α - β - γ Cost Model

- α = per-message latency (startup cost).
- β = per-byte inverse bandwidth (serialisation).
- γ = per-byte reduction-arithmetic cost (usually $\ll \beta$).

One point-to-point send of M bytes: $\alpha + M\beta$.

A collective with s sequential steps of M_s bytes each:

$$T_{\text{coll}} = \sum_s (\alpha + M_s \beta + M_s \gamma).$$

Typical ratios (rule of thumb)

400 Gb/s Ethernet: $\alpha \sim 1\text{--}5 \mu\text{s}$, $\beta \sim 20 \text{ ps/byte}$,

$\gamma \sim 1 \text{ ps/byte}$ on a modern GPU. $\Rightarrow \alpha \gg \beta M$ for $M \lesssim 100 \text{ KB}$; bandwidth dominates above.

Patarasuk–Yuan: The All-Reduce Lower Bound

Theorem (Patarasuk & Yuan [], JPDC 2009)

On a network with N nodes, All-Reduce of M bytes has

$$T_{\text{AR}} \geq 2 \cdot \frac{N-1}{N} \cdot M\beta + 2 \log_2 N \cdot \alpha.$$

The bandwidth term is tight for any graph with a Hamiltonian cycle; the latency term $2 \log_2 N \cdot \alpha$ is matched by recursive-halving-doubling.

Ring All-Reduce hits the bandwidth bound but pays $2(N-1)\alpha$ — **bad at small M** .

Tree All-Reduce hits the latency bound but sends $O(\log N)$ times the bandwidth bound — **bad at large M** .

— **PolarFly's opportunity** —

A multi-tree algorithm can hit *both* bounds *at once* by running k independent pipelines; bandwidth scales by k , latency stays $\Theta(\log N)$.

Ring, Tree, Recursive-Halving-Doubling

Algorithm	Bandwidth term	Latency term	Sweet spot
Ring	$2 \frac{N-1}{N} M\beta$	$2(N-1)\alpha$	huge M
Binary tree	$O(\log N)M\beta$	$2 \log_2 N \cdot \alpha$	tiny M
Rec. halving/doubling	$2 \frac{N-1}{N} M\beta$	$2 \log_2 N \cdot \alpha$	N power of 2
Multi-tree (PolarFly)	$\frac{2}{k} \frac{N-1}{N} M\beta$	$O(\log N)\alpha$	both

$k = \lfloor (q+1)/2 \rfloor$ = number of edge-disjoint spanning trees embeddable in ER_q . $k = 4$ for $q = 7$, $k = 12$ for $q = 23$, $k = 51$ for $q = 101$. This row is the punchline of Chapter 14.

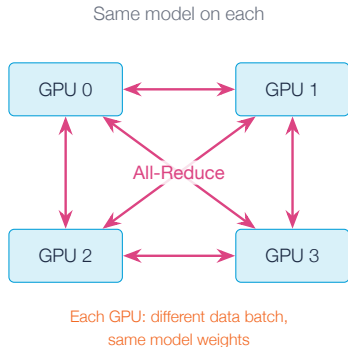
Parallelism Patterns in Distributed Training

Which collective each pattern stresses:

Pattern	Dominant collective
Data Parallel (DP)	All-Reduce on gradient buffers (every step).
Tensor Parallel (TP)	All-Reduce <i>inside</i> every layer's forward/backward pass.
Pipeline Parallel (PP)	Point-to-point; occasional Send/Recv between stages.
Expert Parallel (EP)	All-to-All to route tokens to experts.
FSDP	All-Gather (forward) + Reduce-Scatter (backward).

FSDP and TP dominate large-model training today; both are **sensitive to the bandwidth term** of the collective.

Data Parallel (DP)



Every GPU holds a full copy of the model.

- Each GPU processes a different mini-batch.
- After backward pass: **All-Reduce** to average gradients.
- All replicas apply identical update $\Rightarrow$ stay in sync.

Collective: All-Reduce (every step)
PyTorch DDP [3] wraps the model and inserts All-Reduce hooks automatically. The entire gradient tensor (model-sized) is reduced every iteration.

Tensor Parallel (TP)



Each layer's weight matrix
is split column-wise

All-Reduce *inside* every
forward + backward pass

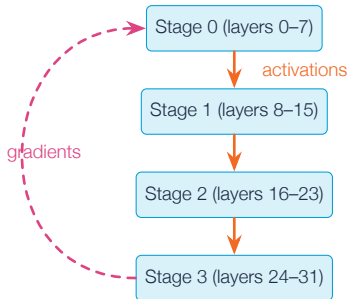
One layer is split across GPUs.

- Weight matrix W is sliced column-wise (Megatron-LM [4] style).
- Each GPU computes a partial output; All-Reduce combines them.
- Happens *twice per layer* (forward + backward).

High frequency, moderate size

TP calls All-Reduce many times per step (once per layer per direction) but each message is only the activation size — much smaller than a full gradient. Latency-sensitive.

Pipeline Parallel (PP)



Micro-batches fill the pipe

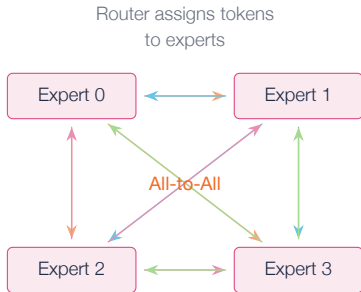
Model is sliced into stages (groups of layers).

- Each GPU runs a contiguous block of layers.
- Activations flow forward; gradients flow backward.
- Communication: **point-to-point** Send/Recv between adjacent stages.
- Micro-batches fill the pipeline to hide bubble time.

Low bandwidth, latency-sensitive

PP sends only activation tensors between neighbors — no collective needed. But the pipeline bubble makes latency critical: every μs of inter-stage delay multiplies by the number of micro-batches.

Expert Parallel (MoE)



Every token goes to its assigned expert

Each GPU hosts a different expert; tokens are routed.

- A learned router picks top- k experts per token.
- **All-to-All** dispatches tokens to expert GPUs.
- After expert computation, a second All-to-All returns results.
- Used in Switch Transformer, Mixtral, DeepSeek-V2.

All-to-All: the hardest collective

Every GPU sends unique data to every other GPU. PolarFly's diameter 2 means every pair is ≤ 2 hops apart — fundamentally better than a Clos's 4–6 hop inter-pod paths.

FSDP (Fully Sharded Data Parallel)



Each GPU stores $1/N$ of params
AG before fwd, RS after bwd

Parameters sharded; reconstructed on the fly.

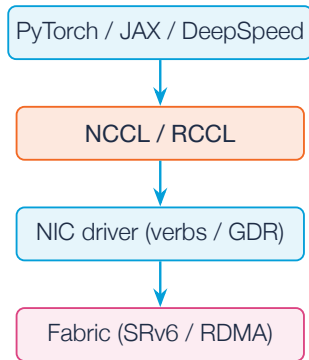
- Each GPU stores only $1/N$ of the parameters.
- **Forward:** **All-Gather** to reconstruct full layer weights, then compute, then discard.
- **Backward:** compute gradients, then **Reduce-Scatter** so each GPU reduces only its shard.
- Memory: $O(M/N)$ per GPU instead of $O(M)$.

Two collectives per layer
FSDP = ZeRO-3. AG + RS = ring All-Reduce — Po-
larFly's multi-tree speed-up applies directly.

What Is NCCL?

NCCL (NVIDIA Collective Communications Library) [5] is the software layer between your training framework and the network hardware.

- Every call to `torch.distributed.all_reduce()` in PyTorch, or the equivalent in JAX / DeepSpeed, lands in NCCL.
- NCCL decides *how* the collective is executed: which algorithm, which path through the fabric, how many parallel streams.
- RCCL (AMD) is the API-compatible equivalent for ROCm GPUs.



Inside NCCL: Channels, Algorithms, Protocols

NCCL organizes every collective into three choices:

Algorithm ring, tree, or collnet (SHARP offload). Each defines a logical communication pattern over the ranks.

Protocol LL (low-latency, 8-byte atomic), LL128 (128-byte chunks), or Simple (large-block copy). Trades off latency vs. bandwidth.

Channels NCCL runs **multiple independent copies** of the chosen algorithm in parallel — each on a separate CUDA stream. More channels = more link utilization, up to the physical bandwidth limit.

NCCL 2.x **auto-selects** algorithm × protocol × channel count based on N , M , and topology hints from `NCCL_TOPO_FILE`.

Why Ring Dominates Today — and What Changes

On today's Clos fabrics:

- Uplinks from leaf to spine are **oversubscribed** (typically 2:1 or 4:1).
- Tree algorithms push traffic *up* through the spine — they hit the oversubscription bottleneck.
- Ring algorithms keep traffic *lateral*, staying within a pod or rack. They avoid the bottleneck.
- Result: **ring wins on Clos** for large messages, despite its $O(N)$ latency.

On PolarFly:

- **No oversubscribed tier** — every link runs at full line rate.
- Tree algorithms use *edge-disjoint* spanning trees: no link contention between trees.
- $k = \lfloor (q+1)/2 \rfloor$ trees run in parallel, each carrying M/k bytes.
- Result: **multi-tree wins on PolarFly** — $k \times$ bandwidth, $O(\log N)$ latency.

Implication for NCCL

A PolarFly-aware NCCL backend declares k channels, one per spanning tree, and runs them in parallel. The existing ring kernel is reused; only the *topology selection* changes.

Worked Example: $q = 7, M = 64$ MB

PolarFly at $q = 7$: $N = 57$ switches.

Pick $\alpha = 2 \mu s$, $\beta = 20$ ps/byte (400 Gb/s).

Algorithm	Bandwidth time	Latency time
Ring All-Reduce	≈ 2.5 ms	$\approx 224 \mu s$
Binary tree	≈ 14 ms	$\approx 24 \mu s$
Rec. halving/doubling	≈ 2.5 ms	$\approx 24 \mu s$
Multi-tree ($k=4$)	≈ 0.63 ms	$\approx 24 \mu s$

Multi-tree: **4× faster** than ring on the bandwidth term, matches rec. halving/doubling on latency. For $q = 23$ ($k = 12$): 12× speed-up. For $q = 101$ ($k = 51$): 51×.

Exercises

1. For $N = 57$ switches and message size $M = 64$ MB with $\beta = 20$ ps/byte, compute the ring All-Reduce bandwidth term $2\frac{N-1}{N}M\beta$.
2. With $k = 4$ spanning trees, recompute the multi-tree bandwidth term $2M\beta/k$ for the same M and β . What is the speedup factor over ring?
3. Identify which collective(s) each LLM parallelism strategy primarily uses: (a) Data Parallel, (b) Tensor Parallel, (c) Pipeline Parallel, (d) FSDP.
4. At what message size M^* does the latency term $2\log_2 N \cdot \alpha$ equal the bandwidth term $2M\beta/k$? Compute for $N = 57$, $k = 4$, $\alpha = 2\mu\text{s}$, $\beta = 20$ ps/byte.

Chapter 3

All Reduce on Disjoint Trees



Multi-Tree All-Reduce: Many Trees in Parallel

Problem. All-Reduce sums N vectors back to all N switches. Classical algorithms drive the entire reduction over *one* structure and pay a price at one end of the size spectrum:

Structure	Latency	Bandwidth	Hurts at
Ring (1 tree)	$2(N-1)\alpha$	$2 \frac{N-1}{N} M\beta$	small M
Binary tree (1 tree)	$2 \log_2 N \alpha$	$O(\log N) M\beta$	large M
Recursive halving	$2 \log_2 N \alpha$	$2 \frac{N-1}{N} M\beta$	non-2-power N
Multi-tree (k disjoint trees)	$d_T \alpha$	$\frac{2}{k} \frac{N-1}{N} M\beta$	neither

Multi-tree, in one sentence. Build k **edge-disjoint** spanning trees and run All-Reduce on *all of them in parallel*, each tree carrying a $1/k$ -slice of the message. Bandwidth divides by k ; latency stays at the depth d_T of one tree.

Two questions, one chapter

(1) How big can k be on ER_q ? (Answer: Nash–Williams / Edmonds.) (2) How do we build the trees? (Answer: Singer rotation, Lakhotia et al.)

Predecessor: PF.TREE2 (Chapter 8 Recap)

What it is. An SRv6 “one-tree” multicast that exploits PolarFly’s diameter-2 property. The source replicates to its $q+1$ neighbours; each hop-1 switch replicates to its q *other* neighbours (split-horizon excludes the back-edge).

Why it works. Coverage in two hops:

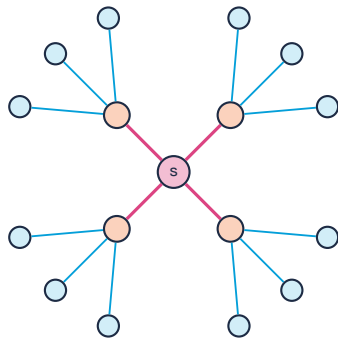
$$1 + (q+1) + (q^2+q) = N + (q+1) - 1,$$

minus duplicates. Girth 5 (Chapter 8) ensures no two hop-1 fan-outs collide, so every one of the $q^2+q+1 = N$ vertices receives the message exactly once.

Cost model.

- Latency: 2α (*constant in N*).
- Bandwidth: $\sim NM\beta$ (every link carries the full message once).

Sweet spot. **Tiny messages**, where the 2α floor dominates
— TREE2 is unbeatable at small M .



$$1 + (q+1) + q^2 = N \text{ in 2 hops}$$

$q = 3$ schematic. Magenta = hop-1 fan; blue = hop-2 fan.

Lakhotia et al. (SPAA'24): Multi-Tree on ER_q

Result (Lakhotia, Besta, Hoefler et al. 2023 [6]). ER_q admits **two explicit multi-tree constructions** with opposite trade-offs. Between them, multi-tree beats TREE2 at every message size above a small threshold:

Construction	Trees k	Depth d	Edge-disjoint?	Wins at
C1 “stars” (low-depth)	q	≤ 3	no (congestion 2)	medium M
C2 “snakes” (Hamiltonian paths)	$\lfloor (q+1)/2 \rfloor$	$(N-1)/2$	yes	large M
(For comparison) PF.TREE2	1	2	n/a (broadcast)	small M

- C1 shares edges across trees (each in ≤ 2 trees) to keep depth ≤ 3 . Bandwidth speed-up: $q/2$.
- C2 hits the **Nash–Williams ceiling** — the most edge-disjoint spanning trees the graph can hold — at the cost of depth $(N-1)/2$. Speed-up: $\lfloor (q+1)/2 \rfloor$.
- TREE2, C1, and C2 coexist in the SRv6 data plane (Chapter 8d); fabrics pick per payload size.

Plan for this chapter

Derive both constructions’ properties from first principles: Singer labels, difference sets, Nash–Williams / Edmonds tree packing, and the pipelined-tree cost model. Explicit C1 / C2 algorithms: Chapter 8c.

What We Still Need to Build Multi-Tree

Chapters 3–8 gave us $\mathbb{F}_q$, $\text{PG}(2, q)$, and ER_q .

Two *algebraic* ingredients & two *graph-theoretic* toolkits before the tree construction:

1. **Singer representation** (algebra). Replace the triple $[z_0 : z_1 : z_2]$ with a single integer $i \in \{0, \dots, N-1\}$. Adjacency becomes “does $i - j$ lie in a fixed set D ?”.
2. **Singer rotation is a graph automorphism** (algebra $\rightarrow$ graph). Adding a constant to every label produces an isomorphic subgraph. Free trees — but not automatically edge-disjoint.
3. **BFS & spanning trees** (graph primer). The basic constructive object: layer-by-layer traversal from a root. Depth is bounded by the graph diameter. On ER_q , diameter = 2.
4. **Matroids & tree packing** (graph primer). The structural reason “how many edge-disjoint spanning trees does G have?” has a clean answer: Nash-Williams/Edmonds/Tutte via matroid union.
5. **Pipelined-tree cost model** (analysis). Why k trees divides the bandwidth term by k , and why depth controls the latency term.

One ingredient per slide — primer style.

We cite Nash-Williams (1961), Tutte (1961), Edmonds (1965 matroid union; 1973 corollary), Menger (1927), and Singer (1938), but prove none of them in full. The deep dives are in the book’s bibliography.

The 30-Second Pitch — Why Multi-Tree At All?

The job. Every All-Reduce in deep learning is the same sentence: “ N machines, each holds a vector, return the *sum* of all the vectors to every machine.”

The trick. If we send everything down *one* spanning tree, only *one* link out of the root is hot at a time. The other q links sit idle.

The fix. Build k **edge-disjoint** spanning trees. Stripe the data k -ways across them. Now k links work in parallel and the bandwidth bill drops by a factor of k .



One tree fires one link at a time — the other q links are idle. Four edge-disjoint trees fire four links in parallel — bandwidth $\times 4$. This is the prize at the end of the math.

What we will learn from the math

- how many such trees a graph can hold (Nash–Williams);
- a recipe for building them on ER_q (Singer rotation);
- the cost equation that says “how much faster?” (α - β pipeline model).

Five slides of math, then we read the speed-up off a table.

Four Theorems That Power Multi-Tree

The multi-tree construction rests on four classical results. Each appears later in this chapter; here is what they say and *why you should care*.

Menger (1927)	Max edge-disjoint paths between two vertices = min cut.	→ PolarFly has $q+1$ disjoint paths between any pair.
Nash-Williams / Tutte (1961)	k edge-disjoint spanning trees exist iff every partition has $\geq k(r-1)$ crossing edges.	→ PolarFly packs $\lfloor (q+1)/2 \rfloor$ trees.
Edmonds (1973)	$2k$ -edge-connected $\Rightarrow k$ trees guaranteed. No partition check needed.	→ ER_q is $(q+1)$ -connected; trees come instantly.
Singer (1938)	Every projective point gets one integer label; $+1$ rotation is a graph automorphism.	→ Rotate one tree to get k trees — explicit, no search.

The chain — Menger → connectivity. Nash-Williams → tree count. Edmonds → easy verification. Singer → explicit construction.

The Singer Representation: One Integer per Point

Theorem (Singer [] 1938, cyclic plane)

Let $\alpha \in \mathbb{F}_{q^3}^*$ be a primitive element. The map

$$\phi : \{0, 1, \dots, N-1\} \rightarrow \text{PG}(2, q), \quad \phi(i) = \alpha^i \pmod{\mathbb{F}_q^*},$$

is a bijection. $i \mapsto i + 1 \pmod{N}$ is a projectivity of order N (the *Singer cycle*).

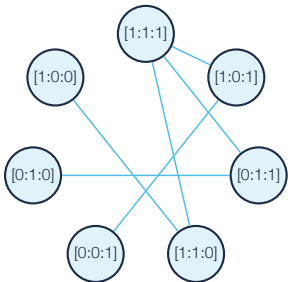
Effect: every point of $\text{PG}(2, q)$ gets a single integer label in $\{0, \dots, N-1\}$, and “next point on the Singer cycle” is just $+1 \pmod{N}$.

$q = 2$ (Fano plane), $N = 7$

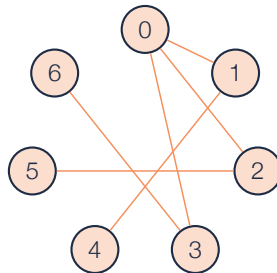
$\mathbb{F}_8 = \mathbb{F}_2[x]/(x^3+x+1)$, $\alpha = x$. The seven points of $\text{PG}(2, 2)$ are labeled $\{\alpha^0, \alpha^1, \dots, \alpha^6\}$, i.e. just $\{0, 1, \dots, 6\}$.

Singer Labels: Same Plane, Two Names

Triples (algebraic name)



Singer integers (combinatorial name)



What changed and what didn't

- **Same graph:** same vertices, edges, adjacency. Singer is a renaming, not a new graph.
- **Smaller name tag:** $[1:1:1]$ takes $3\lceil \log_2 q \rceil$ bits; integer 0 takes $\lceil \log_2 N \rceil$ bits.
- **Adjacency becomes arithmetic:** “do these triples line up?” becomes “is $i-j \in D$?” (next slide).

Three Concepts We Need

The Singer construction uses three ideas from algebra and graph theory. One slide each follows; here is the roadmap.

Residue (mod N) Clock arithmetic. How we test adjacency in the Singer labelling: two vertices i, j are adjacent iff their difference $i - j \pmod{N}$ lands in a fixed set D .

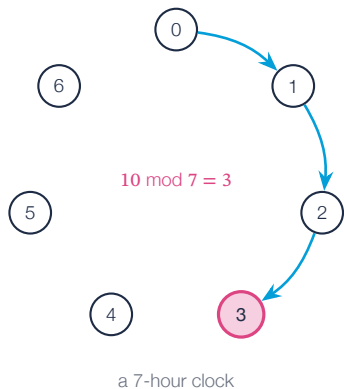
Isomorphism “Same shape, different drawing.” Two graphs are isomorphic if you can rename the vertices of one to match the other edge-for-edge. Key claim: rotating a tree gives an *isomorphic* tree.

Automorphism An isomorphism from a graph to *itself* — a symmetry. Singer’s $+1$ rotation is an automorphism of ER_q : it gives us a fresh spanning tree for free.

Why all three matter

Residue $\Rightarrow$ adjacency test. **Isomorphism** $\Rightarrow$ “same tree, just relabelled.” **Automorphism** $\Rightarrow$ the rotation *is* the isomorphism; a new tree for free.

Residue (mod N)



Definition. $r \bmod N$ is what is *left over* after dividing r by N .

Picture a clock with N hours. Walk forward r ticks from noon — wherever you land is the residue.

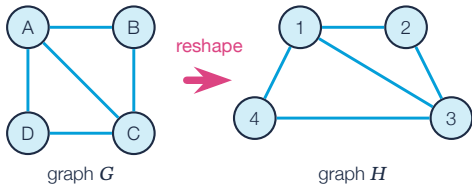
Example. $10 \bmod 7 = 3$: start at 0, walk 10 ticks around a 7-hour clock, land on 3.

“Nonzero residue” just means “not noon (0).”

Why it matters

In the Singer labelling, vertices i and j are adjacent iff $i - j \pmod{N}$ lies in the difference set D . Adjacency is just clock arithmetic.

Isomorphism: Same Shape, Different Drawing



Definition. Two graphs are *isomorphic* if you can rename the vertices of one to make every edge line up with an edge of the other.

G and H look different, but they have the *same* connection pattern under the renaming:

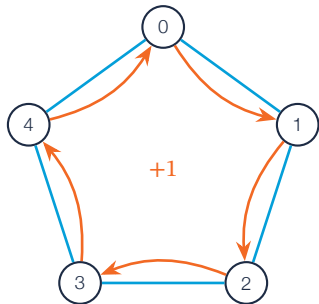
$$A \rightarrow 1, \quad B \rightarrow 2, \quad C \rightarrow 3, \quad D \rightarrow 4.$$

The shape is preserved; only the labels changed.

Why it matters

When we rotate a spanning tree by the Singer map, the result is an *isomorphic* tree — same structure, different vertex labels.
“Same tree, just relabelled.”

Automorphism: A Graph's Symmetry



Definition. An *automorphism* is an isomorphism from a graph to *itself* — a permutation of the vertices that preserves every edge.

Example. “Rotate by +1” on the 5-cycle:
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$. Every edge $i-(i+1)$ maps to the edge $(i+1)-(i+2)$ — still an edge. The graph looks the same after the rotation.

This is a **symmetry**: the graph can’t tell it was moved.

Why it matters

Singer’s +1 map on ER_q is an automorphism. Applying it to a spanning tree produces a *new* spanning tree — a fresh tree for free. Apply it k times $\Rightarrow k$ trees.

Singer Difference Set: Adjacency as Subtraction

Let $D \subseteq \{0, 1, \dots, N-1\}$ be the Singer labels of the $q + 1$ points on one fixed line.

Theorem (Singer difference set)

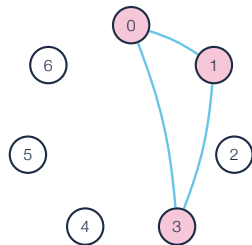
Every nonzero residue $r \pmod{N}$ can be written as $r = d_1 - d_2$ with $d_1, d_2 \in D$ in *exactly one* way.

Corollary (Singer adjacency rule)

i and j are adjacent in ER_q iff $i - j \pmod{N} \in D$.

Adjacency test has collapsed from “compute a dot product in $\mathbb{F}_q^3$ ” to “is $(i - j) \pmod{N}$ in a $(q+1)$ -element set?”.

Example ($q = 2$): $D = \{0, 1, 3\}$ in $\mathbb{Z}/7$. Differences: $1-0=1$, $3-0=3$, $3-1=2$, $0-1=6$, $0-3=4$, $1-3=5$. All six nonzero residues, each appearing *exactly once*.



$D = \{0, 1, 3\}$: highlighted

The three fuchsia points are D ; every other vertex is adjacent to vertex 0 via a unique difference in D .

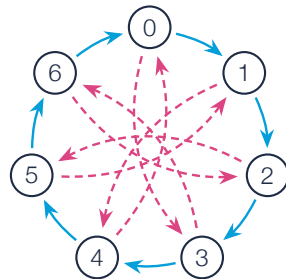
Differences Walked Out: $q = 2$ in Slow Motion

Setup. $N = q^2 + q + 1 = 7$, difference set $D = \{0, 1, 3\}$ (we read this off the Singer construction in the previous slide).

What the theorem promises. Every nonzero residue $r \in \{1, 2, 3, 4, 5, 6\}$ shows up exactly *once* as a difference $d_a - d_b$ with $d_a, d_b \in D$.

Let's check by hand. Pick two elements of D , subtract, take the answer mod 7. Six ordered pairs, six residues:

d_a	d_b	$d_a - d_b \bmod 7$
1	0	1
3	0	3
3	1	2
0	1	6
0	3	4
1	3	5



1. $d=1$: each node $\rightarrow$ next neighbor (7 edges)
 2. $d=3$: each node $\rightarrow$ skip-2 neighbor (7 edges)
- ($d=0$ is the trivial self-identity.) Together: $7+7 = 14$ undirected edges = $\binom{7}{2} \Rightarrow K_7$.

Intuition: Singer +1 Is a Graph Automorphism

Claim. The map $\sigma : i \mapsto i + 1 \pmod{N}$ is a *graph automorphism* of ER_q .

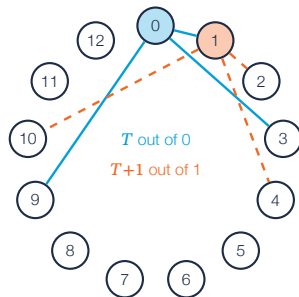
One-line proof. i and j are adjacent

$\iff i - j \in D \iff (i+1) - (j+1) \in D \iff \sigma(i)$ and $\sigma(j)$ are adjacent. $\square$

Why we care. Any subgraph $H \subseteq \text{ER}_q$ has a rotated twin $\sigma(H) = H + 1$ that is *isomorphic* to H . In particular:

- If T is a spanning tree, so is $T + c$ for any c .
- If T has depth d from root r , then $T + c$ has depth d from root $r + c$.
- So we get N “for free” copies of any tree — all with identical depth. Only question: are any of them edge-disjoint?

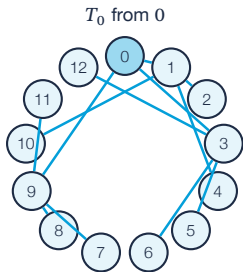
Key fact (next chapter): the rotations $\{T_0, T_{d_1}, T_{d_2}, \dots\}$ by the difference set itself are **not** pairwise edge-disjoint. Finding an edge-disjoint family requires a *different* construction (Hamiltonian paths; Chapter 8c).



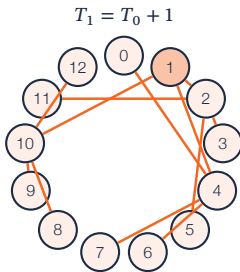
Rotating the three blue edges by $+1$ produces the three dashed orange edges — same shape, different vertices. Blue and orange edge sets are disjoint here; in general, rotates *may* collide.

Rotation in Action: Three Trees from One ($q = 3$)

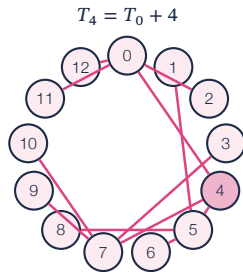
The recipe. Pick a base tree T_0 rooted at vertex 0. Rotate by $+1$ to get T_1 rooted at 1; by $+4$ to get T_4 rooted at 4. Same shape, same depth, different vertices — the Singer cycle moves the whole tree as one rigid object.



root 0, children {1, 3, 9}



root 1, children {2, 4, 10}



root 4, children {5, 7, 0}

What the picture shows

- Each tree spans all 13 vertices, depth 2. Same shape, just rotated.
- “For free”: one BFS $\rightarrow$ 13 trees.
- But: some rotations share edges — the subtlety C2 must work around.

Toolkit: BFS and Spanning Trees, in One Slide

Breadth-First Search (BFS) from root r . Explore G in *layers* of increasing distance from r :

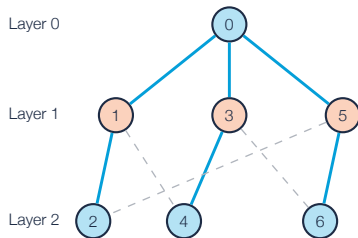
- Layer 0 = $\{r\}$. Mark r visited.
- Layer $\ell + 1$ = every unvisited neighbour of a layer- ℓ vertex.
- For each newly visited vertex v , set its *BFS parent* to the layer- ℓ neighbour that discovered it. (Ties broken arbitrarily.)

Spanning tree T_r . The parent-pointer graph is a tree that touches every vertex: a **spanning tree** of G . Its **depth** is $\text{ecc}(r) = \max_v \text{dist}(r, v)$.

Diameter. $\text{diam}(G) = \max_r \text{ecc}(r)$. Every BFS tree has depth $\leq \text{diam}(G)$.

PolarFly fact (Chapter 8)

$\text{diam}(\text{ER}_q) = 2$. So every BFS tree of ER_q has depth ≤ 2 — we never need three hops to reach any vertex from any root. That is what makes C1's “depth ≤ 3 ” feasible: only the few vertices not reached by the depth-2 Singer algebra need a third hop.



Solid blue = T_0 (BFS tree, 6 edges, depth 2). Dashed grey = non-tree graph edges (would create cycles).

BFS Step-by-Step on the Fano Plane

Goal. Visit every vertex of $ER_q(2)$ starting from vertex 0, recording the edge used to reach each new vertex. Those edges form the BFS tree. Adjacency comes from the difference set $D = \{0, 1, 3\}$, so the neighbours of vertex v are $\{v + 1, v - 1, v + 3, v - 3\} \bmod 7 = \{v \pm 1, v \pm 3\}$.

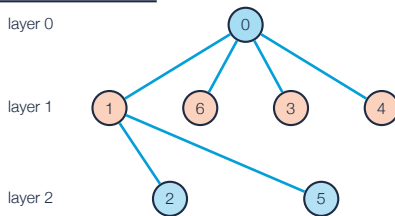
Step	Frontier (queue)	Pop	Add to tree	Edges
1	{0}	0	0-1, 0-6, 0-3, 0-4	4
2	{1, 6, 3, 4}	1	1-2, 1-5 (others taken)	6
3	{6, 3, 4, 2, 5}	6	— (all visited)	6
4	{3, 4, 2, 5}	3	— (all visited)	6

Reading the table

Step 1: pop vertex 0, discover four neighbours. Step 2: pop 1; only two still *unvisited*. Step 3: every vertex seen—queue drains.

What we walk away with

- 7 vertices, 6 tree edges (always $N-1$).
- Depth = 2 (two layers below root).
- One of many BFS trees; tie-breaking changes shape, not depth.



Solid blue = the 6 tree edges. Vertices 2 and 5 end up in layer 2

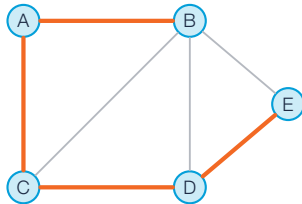
What Is a Matroid?

Definition

A matroid [8] $M = (E, \mathcal{I})$ is a finite ground set E with a family $\mathcal{I} \subseteq 2^E$ of *independent sets* satisfying:

1. Downward closed: $A \in \mathcal{I}, B \subseteq A \Rightarrow B \in \mathcal{I}$.
2. Exchange: $|A| < |B| \Rightarrow \exists e \in B \setminus A$ with $A \cup \{e\} \in \mathcal{I}$.

All maximal independent sets (*bases*) have the same size.



7 edges, 4 in this spanning tree

Independent set = edges with no cycle (forest).

Basis = spanning tree ($n-1$ edges).

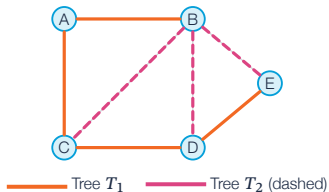
Greedy works: exchange axiom lets us always extend.

Key example: the graphic matroid

Given graph G : $E=E(G)$, $\mathcal{I}$ =forests. Bases = spanning trees ($n-1$ edges). Kruskal's MST = greedy on this matroid.

Forests on ER_q form a graphic matroid $\Rightarrow$ Nash-Williams greedy finds the max edge-disjoint spanning trees.

Tree Packing: From One Tree to k



Two edge-disjoint spanning trees using 7 of 7 edges.

Nash-Williams / Edmonds

k edge-disjoint spanning trees exist iff for every partition of V into r parts, the cross-part edge count is $\geq k(r-1)$.

Matroid union (Edmonds & Fulkerson [9], 1965).

- Take k copies of the graphic matroid $M(G)$.
- Their union $M_1 \vee \dots \vee M_k$ has independent sets = unions of k pairwise-disjoint forests.
- A basis of size $k(n-1)$ is exactly k edge-disjoint spanning trees.

Algorithmic upshot.

- Augmenting-path algorithm (Roskind & Tarjan [10], 1985): $O(k^2 n^2)$ time.
- Generic but expensive for large k .

PolarFly's shortcut

Singer's cyclic construction gives an *explicit* table of $\lfloor (q+1)/2 \rfloor$ edge-disjoint spanning trees — no search needed. Matroid theory says “they exist”; Singer says “here they are.”

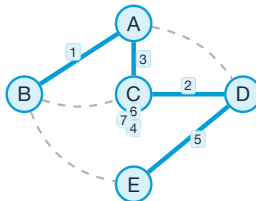
Why Greedy Works: Kruskal in Pictures

Greedy MST in one sentence. Sort all edges by weight, then go down the list and *add the next edge unless it would create a cycle*. Stop when you have $n - 1$ edges.

Sorted edge list:

- | | |
|----------|------------------------------|
| 1. $A-B$ | add (forest size 1) |
| 2. $C-D$ | add (forest size 2) |
| 3. $A-C$ | add (forest size 3) |
| 4. $B-C$ | cycle: skip |
| 5. $D-E$ | add (forest size 4 = $n-1$) |
| 6. $A-D$ | done, skip |
| 7. $B-E$ | done, skip |

MST weight = $1 + 2 + 3 + 5 = 11$. Done in 4 adds and 1 rejection.



kept (4 edges, weight 11)

skipped (cycle / done)

Five nodes, seven edges. Each edge has a *weight* (labeled 1–7; lower = cheaper).

Why this works (matroid exchange property)

Reject-then-swap can never strand you: a heavier tree edge is always replaceable by a cheaper one. Greedy works on every matroid; tree packing on ER_q uses the same lemma.

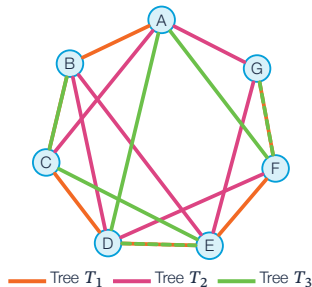
The Goal: Parallel All-Reduce via Tree Packing

- **Goal:** Run All-Reduce across every switch in as few rounds as possible.
- **Method:** Build spanning trees that **share no links**; reduce on each tree **simultaneously**.
- **Payoff:** More disjoint trees = more parallelism = higher bandwidth.

The mathematical question. Given a graph G , how many edge-disjoint spanning trees can you pack into it?

The **tree-packing problem**, solved in 1961:

- **Nash-Williams** (1961) & **Tutte** (1961): existence theorem.
- **Edmonds** (1973): the practical, algorithmic corollary.



Three edge-disjoint spanning trees = three simultaneous All-Reduce channels.

Why this is exciting for PolarFly

ER_q is $(q+1)$ -edge-connected $\Rightarrow$ Edmonds guarantees $\lfloor (q+1)/2 \rfloor$ edge-disjoint spanning trees. At $q=31$: 16 parallel reductions on a 993-switch fabric.

Nash–Williams → Edmonds → PolarFly

Tree packing (Nash-Williams / Tutte, 1961)

G has k edge-disjoint spanning trees iff for every partition of $V(G)$ into r parts, the cross-part edges number $\geq k(r-1)$.

Edmonds' shortcut (1973)

G is $2k$ -edge-connected $\Rightarrow G$ has k edge-disjoint spanning trees.

No partition checking needed — just verify the edge-connectivity of the graph.

The PolarFly payoff

ER_q is $(q+1)$ -edge-connected (every non-quadric vertex has degree $q+1$; the algebraic structure prevents any sparse cut).

By Edmonds: $\lfloor (q+1)/2 \rfloor$ disjoint trees are **guaranteed**.

q	trees k	switches N
7	4	57
23	12	553
101	51	10,303

Each row = k parallel All-Reduces on one fabric.

Nash–Williams: The Cut Test

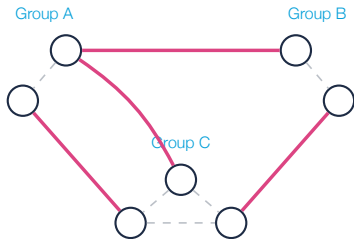
The slogan

A graph holds k edge-disjoint spanning trees iff every way of cutting the vertices into groups leaves enough cross-group edges to wire one spanning tree per group, k times over.

The arithmetic. Cut V into r groups. Count cross-group edges E_{cross} . Then:

k -tree packing exists $\iff E_{\text{cross}} \geq k(r-1)$ for every partition.

Why $r-1$? A spanning tree on r groups needs $r-1$ inter-group edges; we need k copies, all on different physical edges.



$$E_{\text{cross}} = 4, \quad r = 3$$

Reading the diagram

$r = 3$ groups $\Rightarrow$ need $E_{\text{cross}} \geq 2k$. With 4 cross-edges, $k \leq 2$ on this partition. The *tightest* partition across all cuts determines k .

Nash–Williams: Why It's Hard, and the Shortcut

The brute-force problem.

- Nash–Williams asks you to check 2^{N-1} vertex partitions.
- For $N = 57$ (PolarFly at $q = 7$) that's $\approx 7.2 \times 10^{16}$ cuts — astronomical.
- Each cut needs an edge count and a comparison; nobody wants to enumerate.

The point

Nash–Williams converts “can I pack k trees?” into “is there a sparse cut?” A sparse cut is a bottleneck that all k trees would have to fight over.

Edmonds’ shortcut (1973).

- If the graph is $2k$ -edge-connected (every cut has $\geq 2k$ edges), the Nash–Williams test passes *automatically* for every partition.
- Replace exponential enumeration with one connectivity certificate.

Worked example: ER_q at $q = 7$

Given. $N = q^2 + q + 1 = 57$, regular of degree $q + 1 = 8$.

Step 1. Edge-connectivity $\lambda(ER_q) = q + 1 = 8$. (Polarity-graph fact: every cut has at least $q + 1$ edges, the minimum vertex degree.)

Step 2. Pick the largest k with $2k \leq 8$. That's $k = 4$.

Step 3. Edmonds: 8-edge-connected $\Rightarrow$ Nash–Williams passes for all 2^{56} partitions *without checking any of them*.

Conclusion. $ER_q[7]$ packs $\lfloor (q+1)/2 \rfloor = 4$ edge-disjoint spanning trees, guaranteed.

Bottom line

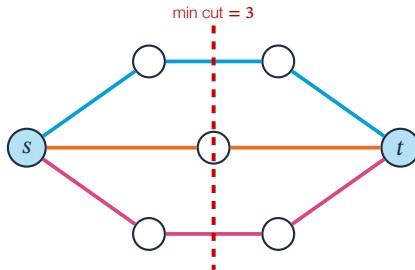
$(q+1)$ -edge-connectivity $\Rightarrow \lfloor (q+1)/2 \rfloor$ edge-disjoint spanning trees $\Rightarrow \lfloor (q+1)/2 \rfloor$ -way parallel All-Reduce on ER_q .

Menger's Theorem: Two Pipes, One Wall

Karl Menger (Vienna, 1927) asked a deceptively simple question about networks: if you want to send as much traffic as possible from s to t , is the answer the same as asking how hard it is to *disconnect* s from t ? Yes—and that equivalence is one of the most useful results in graph theory.

Two questions, same answer.

- Q1. How many *edge-disjoint* paths from s to t ? (How much water can flow at once?)
- Q2. How few edges must I remove to disconnect s from t ? (How thin a wall can stop the flow?)



3 edge-disjoint paths = 3 edges in any s, t cut.

Menger's Theorem [] (1927)

Q1 and Q2 have the *same answer*:

$$\lambda(s, t) = \min \text{cut}(s, t) = \max \text{flow}(s, t).$$

Why we love it

" k -edge-connected" just means $\lambda(s, t) \geq k$ for all pairs s, t . On ER_q , any two vertices have $q+1$ disjoint paths (via common neighbours), so $\lambda(\text{ER}_q) = q+1$. Plug into Edmonds: $\lfloor (q+1)/2 \rfloor$ edge-disjoint spanning trees—the exact number of parallel All-Reduces PolarFly can run simultaneously.

ER_q Carries $\lfloor (q+1)/2 \rfloor$ Trees

- Every vertex of ER_q has degree $q + 1$ (or q for quadrics, with a self-loop convention).
- Any two non-adjacent vertices share q common neighbours (Chapter 8).
- **Menger's theorem** (gloss): the minimum number of edges you must remove to disconnect two vertices equals the maximum number of edge-disjoint paths between them. On ER_q, $q+1$ paths exist (via the q shared neighbours plus the direct edge if any), so ER_q is $(q+1)$ -edge-connected.
- By the Edmonds corollary (slide 8b.4), $(q+1)$ -edge-connectivity $\Rightarrow$ ER_q contains $\lfloor (q+1)/2 \rfloor$ edge-disjoint spanning trees.

q	$N = q^2 + q + 1$	Degree $q+1$	$k_{\max}$	% bw vs. ring
3	13	4	2	50%
7	57	8	4	25%
11	133	12	6	17%

Pipelined-Tree Cost Model

Run All-Reduce on one spanning tree of depth d .

Split M bytes into p chunks of size M/p ; reduce each chunk up the tree, broadcast down.

Pipeline fills after d steps.

$$T_{\text{pipe}}(p) = (p + d - 1) \left(\alpha + \frac{M}{p} \beta \right).$$

Minimize over p : bandwidth term $\sim 2M\beta$, latency term $\sim d \cdot \alpha$. Now with k trees of depth d and edge-congestion c (C1 has $c=2$; C2 has $c=1$), split M across *trees* as well:

$$T_{\text{MT}}(p) = (p + d - 1) \left(\alpha + \frac{cM}{kp} \beta \right).$$

Bandwidth term is divided by k/c ; latency term grows with d .

Pipelining Is Just an Assembly Line

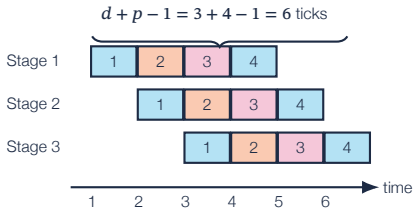
Assembly-line analogy. Imagine a factory with d stations in series. Every car visits every station. If you have p cars, the factory produces them in $d + p - 1$ ticks — d ticks to fill the line, then one car popping out per tick.

Translation.

Factory term	Multi-tree term
<i>Stations</i>	tree depth d (tree levels)
<i>Cars</i>	chunks p of the message
<i>Tick length</i>	$\alpha + \frac{cM}{kp} \beta$ (per-step cost)
<i>Total time</i>	$(p + d - 1) \cdot \text{tick length}$
<i>Parallel lines</i>	k edge-disjoint trees

Two knobs, two regimes.

- Big message, few trees: *bandwidth-bound*. Open more trees ($k \uparrow$) until you hit the Nash–Williams ceiling.



Stage bars show one chunk advancing; four chunks ($p = 4$) pipe through three stages ($d = 3$) in six ticks. Adding a parallel line (another disjoint tree) halves the per-stage bytes.

Depth-Bandwidth Trade-off on ER_q (Two Constructions)

Two competing goals: *many* trees (large k), *shallow* trees (small d). SPAA'23 gives **two distinct constructions** with opposite trade-offs:

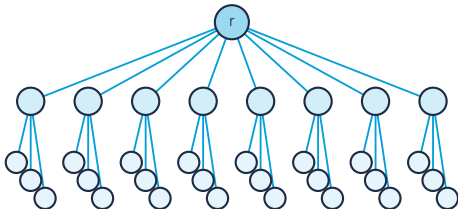
Construction	k (trees)	Depth d	Edge-disjoint?
(C1) Low-depth (Alg. 3)	q	≤ 3	No — congestion 2
(C2) Hamiltonian paths	$\lfloor (q+1)/2 \rfloor$	$(N-1)/2$	Yes
<i>Compare: ring</i>	1	$N-1$	Yes
<i>Compare: rec. halving-doubling</i>	1	$\log_2 N$	N/A

- **C1** minimises *latency*: three-hop trees, but each edge may carry *two* trees (congestion 2), so the bandwidth speed-up is $q/2$, not q .
- **C2** minimises *congestion*: edges fully disjoint across $\lfloor (q+1)/2 \rfloor$ trees (tight Nash-Williams bound), but each tree is a Hamiltonian *path* — depth $(N-1)/2$.
- Implementations mix both: C1 for small payloads (latency- bound), C2 for large payloads (bandwidth-bound).

Plugging into the cost model (one construction at a time):

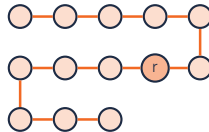
Stars vs Snakes: C1 and C2 at $q = 7$

C1 — “Star” tree, depth ≤ 3



$q = 7$ trees, depth ≤ 3 , but two trees share each edge $\Rightarrow$ congestion 2.

C2 — Hamiltonian “snake” path



$\lfloor (q+1)/2 \rfloor = 4$ trees, all edge-disjoint, depth $(N-1)/2 = 28$ from midpoint r .

Pick by message size

- **Small messages** (latency-bound): **stars (C1)** — the pipe never fills, so the 3-tick depth penalty dominates and you want d small.
- **Large messages** (bandwidth-bound): **snakes (C2)** — the pipe runs at steady state, so the β term dominates and you want full edge-disjointness.
- **Real implementations:** switch automatically based on the byte threshold. Two SRv6 micro-SIDs encode the choice (Chapter 12).

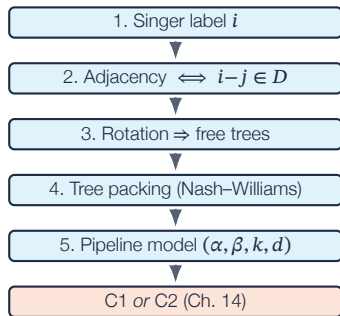
Recap: Five Tools, Two Constructions, One Picture

Five tools in your hands:

1. Singer labels: vertex $\rightarrow$ integer in $\{0, \dots, N-1\}$.
2. Difference set D : adjacency $\iff (i-j) \in D$.
3. Rotation: $+1 \Rightarrow N$ free copies.
4. Matroid greedy: forests form a matroid.
5. Pipeline model: $T = (p+d-1)(\alpha + \frac{cM}{kp}\beta)$.

C1 — Low-depth trees

q trees, depth ≤ 3 , congestion 2. Beats the ring on small payloads.



C2 — Hamiltonian paths

$\lfloor (q+1)/2 \rfloor$ edge-disjoint paths. Tight Nash-Williams. Beats the ring on large payloads.

Exercises

1. Verify the Singer difference set for $q = 2$: show that $D = \{0, 1, 3\} \bmod 7$ produces every nonzero difference exactly once.
2. Prove that the rotation $\sigma : i \mapsto i + 1 \pmod{N}$ preserves adjacency in the Singer labeling (i.e. if $i \sim j$ then $\sigma(i) \sim \sigma(j)$).
3. State the Nash-Williams/Edmonds theorem in one sentence. Then: for $q = 7$, how many edge-disjoint spanning trees does $ER_q[7]$ guarantee?
4. **(Harder.)** For the pipeline model $T = (p + d - 1)(\alpha + \frac{cM}{kp}\beta)$, find the optimal number of pipeline stages p^* that minimizes T as a function of $d, k, c, M, \alpha, \beta$.

Chapter 4



All-Reduce on Multiple Spanning Trees

Collectives on PolarFly – Optimizing Performance

Based on results from the ACM Symposium on Parallelism in Algorithms and Architectures (SPAA) 2024.

Lakhotia *et al.* [6], SPAA 2024

ER_q admits **two explicit spanning-tree constructions** with opposite trade-offs:

C1 (low-depth) q spanning trees, each of depth ≤ 3 , *not* edge-disjoint — every edge of ER_q carries at most 2 trees (“congestion 2”).

C2 (Hamiltonian) $\lfloor (q+1)/2 \rfloor$ edge-disjoint spanning trees, each a Hamiltonian path of depth $(N-1)/2$ rooted at its midpoint.

Pipelined All-Reduce on k trees multiplies the bandwidth term of Patarasuk–Yuan by roughly $1/k$ (after accounting for congestion).

Three moving parts, three slides: (1) the low-depth base tree T_0 (C1); (2) Singer rotations yielding edge-disjoint paths (C2); (3) bandwidth analysis combining both constructions. Then: what this demands from the data plane — motivating the three ASTR profiles of Chapter 15.

Why k Trees? The Big Deal, In One Picture

Claim (intuitive). All-Reduce on one spanning tree uses exactly one “channel” — at any instant, each switch is either forwarding up toward the root or sending the reduction back down.

But ER_q is dense: each switch has $q+1$ neighbors and every link is a real 400 Gb/s pipe. A single tree leaves *most of them idle*.

Idea. Build k spanning trees that don't share edges and run All-Reduce on all of them *at once*. Each chunk of data travels one tree; k chunks finish in parallel.



Illustrative: same 7-node graph, one tree (top) vs. four edge-disjoint trees (bottom). Real ER_q construction comes from Singer rotations — next slides.

Payoff

Bandwidth of All-Reduce drops by a factor of $k = \lfloor (q+1)/2 \rfloor$. At $q = 23$ that's 12x cheaper. The hardware is already there — we're just using it.

Baseline: One BFS Tree on ER_q Wastes Bandwidth

Root r , BFS tree T_r of depth ≤ 2 (diameter 2).

Pipelined All-Reduce on T_r :

$$T_{\text{one-tree}}(p) = (p + 1)\left(\alpha + \frac{M}{p}\beta\right) \sim 2\sqrt{M\alpha\beta} + 2M\beta.$$

- T_r uses $N - 1 = q^2 + q$ edges.
- ER_q has $(q + 1)(q^2 + q + 1)/2 \approx q^3/2$ edges.
- $\Rightarrow \sim q^3/2 - q^2$ edges sit idle.

Opportunity

Route k independent All-Reduces on edge-disjoint trees simultaneously. Aggregate bandwidth $\times k$ at zero hardware cost — we are just using links that were already there.

Construction 1: The Low-Depth Base Tree T_0

Construction 1 (Algorithm 3 in SPAA'23 §3.2) gives q trees, each of depth ≤ 3 , at the cost of *sharing each edge between at most two trees* (congestion 2).

Base tree T_0 (Singer root 0)

Work in Singer labels with adjacency set $D = \{d_0, \dots, d_q\}$:

- Depth 1: the $q + 1$ children are $\{d_0, \dots, d_q\} = D$.
- Depth 2: each $d_s \in D$ gets children $\{d_s + d_t \pmod{N} : d_t \in D, d_s + d_t \neq 0, d_s + d_t \notin D\}$.
- Depth 3: any residual vertex (unreachable in 2 D -steps) is attached via one feasible edge.

By the Singer difference-set property, *almost every* vertex $i \neq 0$ has a unique decomposition $i = d_s + d_t$, so the depth-3 tail has size $O(q)$ — a small fraction of $N = \Theta(q^2)$.

Depth is the point

We accept congestion 2 (each edge carries 2 of the q trees) in exchange for depth ≤ 3 everywhere. This makes Construction 1 ideal for *latency-bound* payloads.

Construction 2: Edge-Disjoint Hamiltonian Paths

When we insist on **edge-disjoint** trees (congestion = 1) the depth-3 structure of Construction 1 cannot survive. SPAA'23 §3.3 gives an explicit family:

Theorem (Lakhotia et al. 2023, §3.3)

On the Singer graph $S_q \cong \text{ER}_q$, there exists a set $M^* \subseteq \{1, \dots, N-1\}$ of size $k = \lfloor (q+1)/2 \rfloor$ such that the **alternating-sum paths**

$$P_m : v \rightarrow v + d_0 \rightarrow v + d_0 - d_m \rightarrow v + 2d_0 - d_m \rightarrow \dots$$

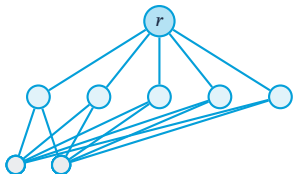
for $m \in M^*$ are pairwise edge-disjoint Hamiltonian paths of ER_q .

Each P_m is a spanning tree of ER_q when viewed as a linear chain; rooted at either endpoint it has depth $N-1$; rooted at the *midpoint* it has depth $(N-1)/2$. Selecting M^* : pairs (d_0, d_m) whose differences generate distinct residues modulo N ; a cheap greedy scan suffices for every prime $q \leq 127$ (proven in the paper).

Intuition — Two Ways to Cover a Graph with Trees

Imagine you need to deliver a message to *every* switch in the fabric. You can build spanning trees and send one copy down each. Two natural strategies:

Strategy 1 — Short & wide (“star”)



Depth ≤ 3 : message reaches everyone fast.

But: different trees rooted at different nodes inevitably *share* some edges — they can't all run at full speed simultaneously.

Strategy 2 — Long & thin (“snake”)



Hamiltonian path: visits every node exactly once.
Each path uses a *unique* set of edges — multiple snakes can run *simultaneously* without contention.

But: depth is $\sim N/2$ — message takes many hops to reach the far end.

The fundamental trade-off

Low depth (fast delivery) vs. **edge disjointness** (full parallelism). You can't have both — this is why PolarFly has two constructions: C1 (stars) and C2 (snakes).

The Math — Edge Budget and Depth Cost

How many disjoint trees can we pack?

ER_q has $N = q^2 + q + 1$ vertices and $|E| \approx \frac{N(q+1)}{2}$ edges. Each spanning tree uses $N-1$ edges, so k disjoint trees need $k(N-1)$ edges:

$$k_{\max} = \left\lfloor \frac{q+1}{2} \right\rfloor.$$

This matches Nash–Williams exactly — **Construction C2 is tight**.

What does a depth- ≤ 3 tree cost?

Root r has $q+1$ children at depth 1. Each child has $\leq q$ non-root neighbours, so depth 2 can reach $(q+1)q = q^2 + q > q^2 - 1$ vertices — enough for *most* of the graph.

But different children's neighbour sets **overlap**: any two depth-1 nodes share q common neighbours.

Why overlap forces edge sharing.

A depth- ≤ 3 tree must assign each depth-2 vertex to exactly one depth-1 parent. When we *rotate* the tree (Singer +1), a different root picks different depth-1 children — but the overlapping neighbour sets mean some depth-2 edges get claimed by both trees.

Consequence

Low-depth trees *cannot* be globally edge-disjoint. Two trees rooted at different vertices will always share $\Omega(q)$ edges.

This is why we need two constructions

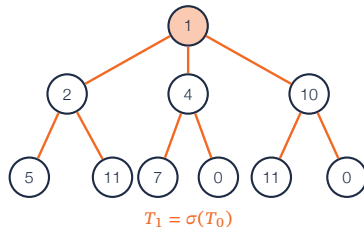
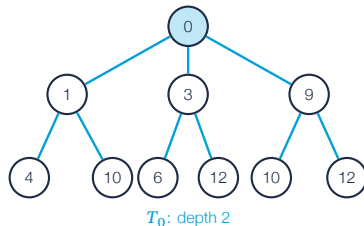
C1: q low-depth trees, congestion 2 (edge sharing tolerated). Best for small M .

C2: $\lfloor (q+1)/2 \rfloor$ edge-disjoint Hamiltonian paths. Best for large M .

Picture: T_0 and T_1 at $q = 3$, Singer Coordinates

$N = 13$, difference set $D = \{1, 3, 9\}$, $k = \lfloor (q+1)/2 \rfloor = 2$.

- T_0 (root 0): children $\{1, 3, 9\}$; each child gets 2 grandchildren via adding another $d \in D$.
- T_1 (root 1 = Singer rotation by +1): children $\{2, 4, 10\}$; same shape as T_0 .
- The two trees share **no edges** — verify by listing their edge sets below.



Edge lists (verify disjointness by eye)

T_0 : $\{0-1, 0-3, 0-9, 1-4, 1-10, 3-6, 3-12, 9-10, 9-12\}$

T_1 : $\{1-2, 1-4, 1-10, 2-5, 2-11, 4-7, 4-0, 10-11, 10-0\}$

(Illustrative — real Singer tree uses the greedy rotation picker from the last slide; numbers shown for a canonical $D = \{1, 3, 9\}$.)

Examples: $q = 3$ and $q = 7$

q	$N = q^2 + q + 1$	k	Tree edges used	ER_q edges
3	13	2	$2 \cdot 12 = 24$	26
7	57	4	$4 \cdot 56 = 224$	228
11	133	6	$6 \cdot 132 = 792$	798
23	553	12	$12 \cdot 552 = 6624$	6636

Saturation ratio “tree edges / ER_q edges” is always $\geq 99\%$. The k trees *nearly fill* ER_q ; only a few edges are left unused. For $q = 3$: tree 1 rooted at 0 with children $\{1, 3, 9\}$; tree 2 is the Singer rotation by 4, rooted at 4 with children $\{5, 7, 0\}$. The 8 edges are pairwise distinct.

How Multi-Tree Pipelined All-Reduce Works

Single-tree pipeline (Patarasuk & Yuan [2], JPDC 2009).

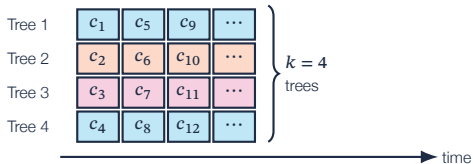
1. Split the M -byte buffer into p chunks of size M/p .
2. Reduce chunk 1 up the tree (depth d hops), then broadcast back down (d hops). While chunk 1 descends, chunk 2 starts ascending — the pipe fills after d steps.
3. Total: $(p + d - 1)$ phases, each costing $\alpha + \frac{M}{p}\beta$.

$$T_{\text{pipe}}(p) = (p + d - 1) \left(\alpha + \frac{M}{p} \beta \right).$$

Multi-tree extension. With k edge-disjoint spanning trees, assign each chunk to one tree — so k chunks are in flight simultaneously on *different links*. Each tree carries M/k bytes:

$$T_{\text{MT}}(p) = (p + d - 1) \left(\alpha + \frac{M}{kp} \beta \right).$$

Bandwidth $\div k$; latency unchanged.



Chunks are round-robin assigned to trees. Within each tree, chunks pipeline through the reduce-up / broadcast-down phases. All k trees operate on *disjoint links*, so they run simultaneously with no contention.

References

Pipeline: Patarasuk & Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," JPDC 2009.

Multi-tree on ER_q: Lakhota *et al.*, SPAA 2024.

Lower bound: $2 \frac{N-1}{N} M\beta$ (Patarasuk & Yuan); multi-tree approaches $M\beta/k$ per tree.

Bandwidth Analysis: C1 vs. C2

The pipelined multi-tree cost model with k trees of depth d and edge congestion c (effective per-tree bandwidth is $1/c$ of the link):

$$T_{MT}(p) = (p + d - 1) \left(\alpha + \frac{cM}{kp} \beta \right).$$

Plugging in each of the paper's constructions:

Algorithm	k	d	c	Bandwidth
Ring	1	$N-1$	1	$2M\beta$
Rec. halv./doub.	1	$\log_2 N$	1	$2M\beta$
Single BFS / ER_q	1	2	1	$2M\beta$
C1 (low-depth)	q	3	2	$4M\beta/q$
C2 (Hamiltonian paths)	$\lfloor (q+1)/2 \rfloor$	$(N-1)/2$	1	$4M\beta/(q+1)$

- C1 and C2 have essentially the **same bandwidth** (both $\sim 2M\beta/k_{\text{eff}}$ with $k_{\text{eff}} \approx (q+1)/2$).
- C1 wins on **latency** (3 vs. $\sim N/2$ hops).
- C2 wins on **link utilisation** (edges fully consumed; no congestion).

Worked Example: $q = 7, M = 256$ MB

$N = 57$. $\alpha = 1 \mu\text{s}$, $\beta^{-1} = 50$ GB/s (400 Gb/s Ethernet).

Single pipelined BFS tree ($k=1, d=2, c=1$)

$$T \approx 2\alpha + 2\sqrt{2M\alpha\beta} + 2M\beta \approx 2 \mu\text{s} + 144 \mu\text{s} + 10.24 \text{ ms} \approx 10.4 \text{ ms}.$$

C1 (low-depth): $k=q=7, d=3, c=2$

Effective $k_{\text{eff}} = k/c = 3.5$; latency term $\sim 180 \mu\text{s}$; bandwidth term $2cM\beta/k = 2.93 \text{ ms}$. Total $\approx 3.1 \text{ ms}$.

C2 (Hamiltonian): $k=\lfloor(q+1)/2\rfloor=4, d=(N-1)/2=28, c=1$

Bandwidth term $2M\beta/k = 2.56 \text{ ms}$; latency term dominated by d but absorbed by pipelining for large M . Total $\approx 2.7 \text{ ms}$.

Ring for comparison

$$T_{\text{ring}} \approx 2(N-1)\alpha + 2M\beta \approx 112 \mu\text{s} + 10 \text{ ms} \approx 10.1 \text{ ms}.$$

Both C1 and C2 give $\sim 3.5\times$ speed-up over single-tree/ring at $q = 7$; which is better depends on M : C1 wins below the pipeline-fill knee, C2 wins above it.

Where the Data Plane Gets Complicated

The SPAA'23 paper answers the *algorithmic* question.

The *runtime* question remains: how does each switch know which tree a given packet is on, and how does it forward it? Three answers, three trade-offs:

Profile	Packet carries	Switch state
A (Algebraic)	algebraic operand	D only
B (Balanced)	tree-set ID + copy selector	replication group table
C (HW-efficient)	global tree ID	per-tree next-hop

The next chapter

Chapter 15 gives full packet formats, local behaviors, and vendor feasibility for all three profiles, based on the ASTR_ARCHITECTURE.md specification.

Exercises

1. For $q = 3$ ($N = 13$), how many edge-disjoint spanning trees does the Nash-Williams bound guarantee? State both the formula and the numeric answer.
2. Explain the C1 tree construction in one paragraph: what is the root, how are depth-1 children chosen, and how are depth-2 children computed?
3. Why can't all k trees in the C1 construction be edge-disjoint? What is the congestion factor, and why is it acceptable in practice?
4. Compare C1 and C2 in a table with columns: number of trees, depth, edge-disjoint?, congestion, best regime (latency-bound or bandwidth-bound).

Chapter 5



ASTR-SRv6: Collective Routing on PolarFly

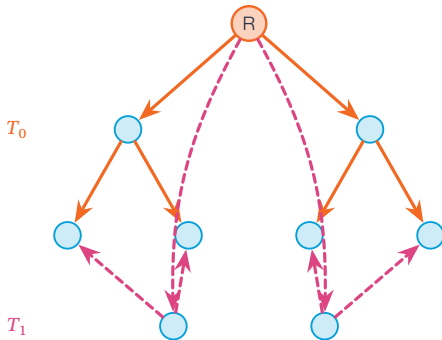
Introducing ASTR

Algebraic Spanning-Tree Routing

The problem. Chapters 12–14 proved that PolarFly admits $\lfloor (q+1)/2 \rfloor$ edge-disjoint spanning trees, enabling a $k\times$ bandwidth speed-up for All-Reduce.

But a mathematical theorem is not a packet format. *How does a switch know which tree a packet belongs to? How does it forward the packet to the right child?*

No existing protocol answers this. NCCL uses rings. MPI uses hand-built trees. Neither can express PolarFly's algebraic structure on the wire.



Two trees, one root, disjoint edges.

Each tree carries a different chunk of the All-Reduce. ASTR tells each switch *which tree* and *which child*.

ASTR: A Cisco Innovation

— ASTR is a Cisco innovation —

ASTR is a purpose-built SRv6 runtime, designed at Cisco, that turns the multi-tree theorem into a deployable forwarding plane. It is not an adaptation of an existing protocol — it is an original Cisco contribution to collective routing on PolarFly.

Three profiles — same packet framework, different silicon trade-offs:

Profile A — Algebraic. Packet carries a math rule; switch computes the next hop on the fly. No per-tree FIB. Requires programmable silicon or SmartNIC.

Profile B — Balanced. Switch-assisted replication via CUD (Carry-on User Data). Moderate FIB: $O(k \cdot |\text{Stages}|)$ entries. Targets Jericho-class / Silicon One (P).

Profile C — Hardware-Efficient. Explicit per-tree FIB compiled by a controller. Runs on **any** SRv6-capable switch today. **Recommended first deployment.**

Profile C ships first; B and A follow as silicon matures.

Why We Need a Runtime, Not Just an Algorithm

Chapter 14: ER_q admits *two* explicit multi-tree constructions (SPAA'24 [6]):

- C1 (low-depth): q trees, depth ≤ 3 , congestion 2.
- C2 (Hamiltonian): $\lfloor (q+1)/2 \rfloor$ edge-disjoint trees, depth $(N-1)/2$.

Running either family in parallel gives a $\sim (q+1)/2 \times$ bandwidth speed-up for pipelined All-Reduce. Runtime question: **how does each switch know which tree a packet is on, and how does it forward it?**

- Encode the tree explicitly (per-tree FIB).
- Carry an *algebraic rule* (no FIB).
- Something in between.

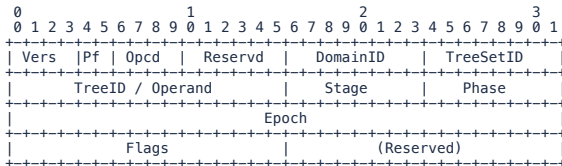
ASTR-SRv6

A family of three SRv6 profiles — A (Algebraic), B (Balanced), C (Hardware-Efficient) — with a *shared* packet framework and three new endpoint behaviors. Protocol efficiency vs. silicon burden trade-off.

Common ASTR Packet Context (WIP)

Every ASTR packet carries a 14-byte context (= 2 uSID slots, Ch. 11, §11.3).

Field	W.	Meaning
Vers	4 b	ASTR version (0x1)
Prof	2 b	A / B / C
Opcode	4 b	ALG_PATH / ROOT_REPL PATH / EGRESS
DomainID	8 b	Tenant / job tag
TreeSetID	8 b	Compiled tree family (B, C)
TreeID/Op	16 b	Tree ID (C) or operand (A, B)
Stage	8 b	Collective stage index
Phase	8 b	Phase / operand selector
Epoch	32 b	Tree-set generation
Flags	16 b	Loop-detect, future use



112 active bits + 16 reserved $\Rightarrow$ 16 B word-aligned; the wire form occupies 14 B (two uSID slots) by packing Flags adjacent to Epoch.

Three new SRv6 behaviors

- End.ASTR.RootReplicate (0xF030) — ingress: resolve tree set, replicate, stamp CUD.
- End.ASTR.Path (0xF031) — transit: resolve next hop from tree context.
- End.ASTR.Egress (0xF032) — terminal: deliver to endpoint.

Profile A — Algebraic ASTR

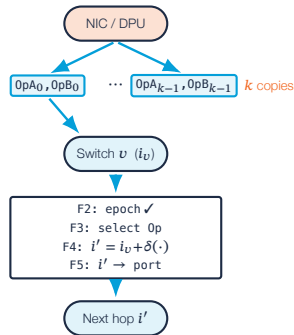
Goal. Packet carries an *algebraic rule*; switch computes next hop on the fly. **No per-tree FIB.**

Forwarding steps at node v (Singer label i_v):

- F1 Parse ASTR context from uSID container.
- F2 Validate Epoch matches local epoch.
- F3 Select operand: $o \leftarrow \text{Phase} ? 0pB : 0pA$.
- F4 Compute next Singer label:
 $i' \leftarrow i_v + \delta(o, i_v \bmod r) \pmod{N}$
 $(\delta: \text{precompiled } O(q)\text{-entry lookup table.})$
- F5 Resolve $i' \rightarrow$ egress port via neighbor-label map ($q+1$ entries).
- F6 Rotate Phase; write back to packet.
- F7 Forward.

State per switch: i_v (own label), neighbor map ($q+1$ entries), δ table ($O(q)$).

Replication: NIC/DPU emits k copies, each with a different $0pA/0pB$ pair.



Hardware assessment

Needs: ALU add-mod- N , δ SRAM (≤ 256 entries).

Risk: highest — programmable switch/DPU. Phase 3.

Profile B — Balanced ASTR (Jericho/SiOne-friendly)

Goal. Switch-assisted replication with modest state.

Ingress (End.ASTR.RootReplicate, root switch):

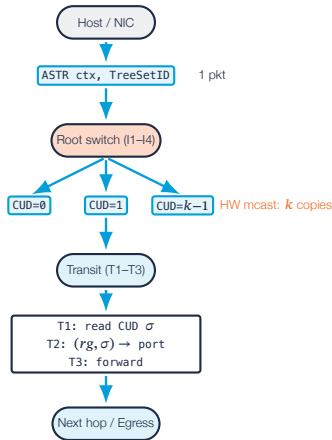
- I1 Parse ASTR context; validate Epoch.
- I2 (TreeSetID , Stage) $\rightarrow$ *ReplicationGroup* rg .
- I3 For each tree $T_m \in rg$, emit one copy with CUD = σ_m (4–8 bits).
- I4 Rewrite active uSID to End.ASTR.Path; forward all copies.

Transit (End.ASTR.Path, every downstream hop):

- T1 Read CUD σ from packet metadata.
- T2 Lookup: (rg , σ , Epoch) $\rightarrow$ (TreeID , NextHop).
- T3 Forward on resolved egress port.

State per switch: compiled table $O(k \cdot |\text{Stages}|)$ — a few kilobytes.

Replication: done *on the switch* at the tree root, using the hardware multicast engine (one packet in, k out).



Hardware assessment

Needs: HW multicast with per-copy CUD (native on Jericho).

Risk: medium — replication memory must be sufficient. Phase 2.

Profile C — Hardware-Efficient ASTR

Goal. Minimize special switch behavior. **Recommended first deployment.**

Fanout (two sub-profiles):

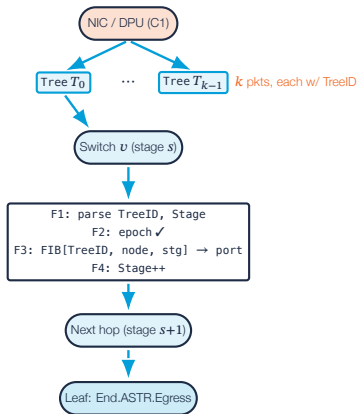
- C1 **Source-side.** NIC/DPU emits one packet per tree (k packets total). Switch ASIC never replicates.
- C2 **Fixed-position.** Root switch replicates; copy position $\Rightarrow$ tree identity.

Forwarding steps (transit, every hop):

- F1 Parse ASTR context; read TreeID and Stage.
- F2 Validate Epoch.
- F3 FIB lookup:
(TreeID, LocalNodeID, Stage) $\rightarrow$ EgressPort.
- F4 Increment Stage.
- F5 Forward.

State per switch: one FIB entry per tree per stage
 $= O(k \cdot \text{depth}) \approx O(q)$ entries.

Replication: NIC/DPU (C1) or root multicast (C2).



Hardware assessment

Needs: standard LPM FIB + counter increment. No ALU, no CUD.

Risk: **lowest** — runs on any SRv6+uSID ASIC. Phase 1.

Profile Comparison

	A (Alg.)	B (Bal.)	C (HW-eff.)
Protocol efficiency (wire state)	best	good	acceptable
Hardware complexity	highest	moderate	lowest
FIB size	$O(q)$ neighbour+ δ	$O(k \cdot \text{Stages})$	$O(k \cdot \text{Stages})$
Replication	NIC/DPU	switch (CUD)	NIC/DPU (C1) or fixed-pos (C2)
Vendor realism (public docs)	DPU/NIC	Jericho/SiOne (P)	all vendors
Deployment risk	highest	medium	lowest
Recommended phase	research (3)	optimize (2)	deploy (1)

Staged deployment

Phase 1. Profile C1 on any SRv6-capable silicon.

Phase 2. Profile B on Jericho-class switches.

Phase 3. Profile A on programmable silicon / SmartNICs.

Worked Example: $q = 3$, Profile C1

ER_q : $N = 13$, $k = 2$ trees, $D = \{1, 3, 9\}$.

Setup: LB = fcbb:bb01::/32, TreeID = 0x1000+ index,
Epoch = 0x0001.

NIC emits 2 packets (Profile C1: source fanout):

Pkt 1 (T_0 , TreeID = 0x1000):

DA = fcbb:bb01:f031:1000:0000:0001:0000:ff00

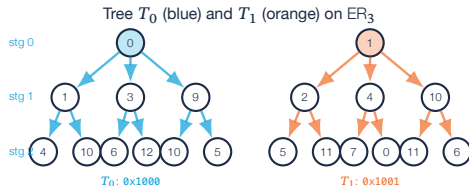
Pkt 2 (T_1 , TreeID = 0x1001):

DA = fcbb:bb01:f031:1001:0000:0001:0000:ff00

fcbb:bb01	LB (PolarFly domain)
f031	End.ASTR.Path
1000	TreelD
0000	Stage:Phase
0001	Epoch
ff00	End-of-Container

DA fields (left to right):

At each hop: switch reads TreeID & Stage, looks up FIB[T , node, stg] $\rightarrow$ port, increments Stage, forwards.

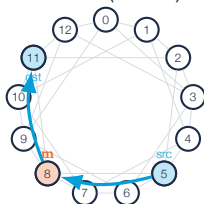


Takeaway

Two packets, two disjoint trees, same SRv6 fast path. Each switch does one FIB lookup per packet — no new microcode, only controller-installed entries.

Unicast vs. ASTR on ER₃: One Picture

Unicast (Ch. 11)



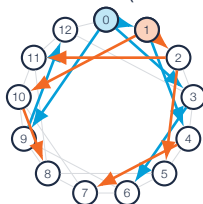
1 packet, 2 hops ($s \rightarrow m \rightarrow d$).

DA: 2 uSIDs (intermediate + destination).

Switch: pop leading uSID, forward.

No FIB beyond neighbor-label map.

ASTR Profile C1 (All-Reduce)



$k = 2$ packets, each on its own tree.

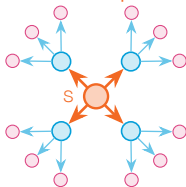
DA: TreeID + Stage; switch looks up $\text{FIB}[T, \text{node}, \text{stg}] \rightarrow \text{port}$.

All 13 nodes via 2 disjoint spanning trees.

	Unicast	ASTR C1
Packets / op	1	k (one per tree)
Hops / packet	2	tree depth (≤ 3)
uSID payload	Cm:Sm + Cd:Sd	TreeID + Stage + Epoch
Switch datapath	uSID pop	$\text{FIB}[T, \text{node}, \text{stg}] \rightarrow \text{port}$
New microcode	none	none (C1)

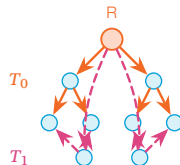
ASTR vs. the Two-Hop Flood: Complementary

TREE2: 2-hop flood



All N switches in 2 rounds

ASTR: k pipelined trees



k trees, each carries M/k bytes

TREE2 floods all switches in ≤ 2 hops — lowest latency, but every link carries the full message.

ASTR pipelines across k edge-disjoint trees — highest bandwidth, but depth-limited latency.

They serve different message-size regimes.

Metric	TREE2	ASTR-SRv6
Latency	2α	$\text{depth}(T)\alpha$
Bandwidth	$NM\beta$	$2M\beta/k$
Best at	small M (latency)	large M (bandwidth)
Use case	barrier, small AR	gradient sync, FSDP

Production fabric uses both

TREE2 = small- M fast path; ASTR Profile C1 = large- M bulk path. NCCL's auto-selector switches at the cross-over message size.

Exercises

1. List the fields a transit switch must read from the ASTR context for Profile A. How many bytes of per-switch state does Profile A require?
2. Why does the ASTR header include an **Epoch** field? What failure mode does it prevent?
3. At what message size M does ASTR's bandwidth advantage ($2M\beta/k$) overtake TREE2's latency advantage (2α)? Compute for $k = 4$, $\alpha = 2\mu\text{s}$, $\beta = 20\text{ ps/byte}$.
4. Compare Profiles A, B, and C in a table with columns: switch state, FIB entries, hardware requirement, and deployment complexity.